

Implementation and Analysis of Fixed-Point FIR Low Pass Filter

Rajesh Reddy

EE24BTECH11043

Department of Electrical Engineering
IIT Hyderabad

1 Introduction

This experiment focuses on the implementation of a 101-tap linear phase FIR low-pass filter using fixed-point arithmetic. The objective is to study architectural trade-offs, pipeline optimization, and hardware efficiency.

Three FIR architectures were implemented:

- Direct Form FIR
- Symmetry-based Adder Tree (Genvar)
- Optimised Symmetric FIR

Both non-pipelined and pipelined versions were designed and analyzed. Further, FIR IP cores were used to compare industry-grade implementations with custom RTL designs.

2 FIR Filter Theory

The FIR filter is defined as:

$$y[n] = \sum_{k=0}^{100} h[k]x[n-k]$$

For linear phase filters:

$$h[k] = h[100 - k]$$

This symmetry enables reduction in computational complexity by pairing terms.

3 Fixed-Point Design

The filter uses fixed-point representation for hardware efficiency.

3.1 Input Representation

- Format: Q3.14
- Total width: 17 bits

3.2 Arithmetic Growth

- Multiplication produces higher precision values
- Accumulation requires wider bit-width to prevent overflow

3.3 Accumulator Design

- 41-bit signed accumulator used
- Maintains precision without truncation

4 Architectures

4.1 1. Direct Form FIR

- Uses all 101 multipliers
- Straightforward implementation
- High area and power consumption
- Large combinational delay

4.2 2. Symmetry-Based Adder Tree

- Exploits coefficient symmetry
- Reduces multipliers to 50
- Uses balanced adder tree
- Improves hardware efficiency

4.3 3. Optimised Symmetric FIR

- Uses sequential accumulation
- Minimizes hardware resources
- Trades speed for area efficiency

5 Pipelined Architecture

Pipelining was introduced to improve timing performance.

5.1 Concept

- Registers inserted between computation stages
- Reduces critical path delay
- Enables higher clock frequency

5.2 Impact

- Increased latency
- Improved throughput
- Better timing closure in FPGA

5.3 Pipeline in FIR

- Multiplication stage registered
- Adder tree split into multiple pipeline levels
- Center tap aligned using pipeline registers

6 Architecture Diagrams

6.1 Direct Form FIR (Pipelined)

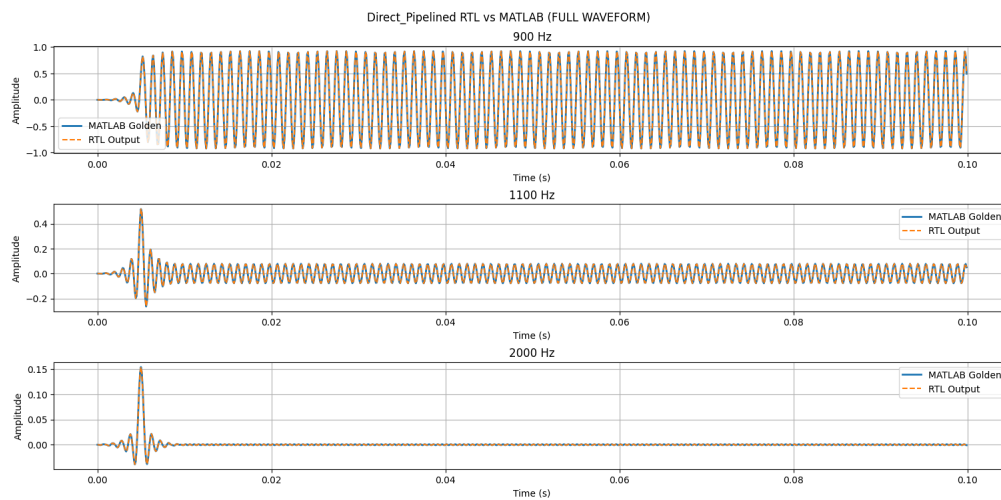


Figure 1: Pipelined Direct Form FIR Architecture

6.2 Symmetry-Based FIR (Genvar)

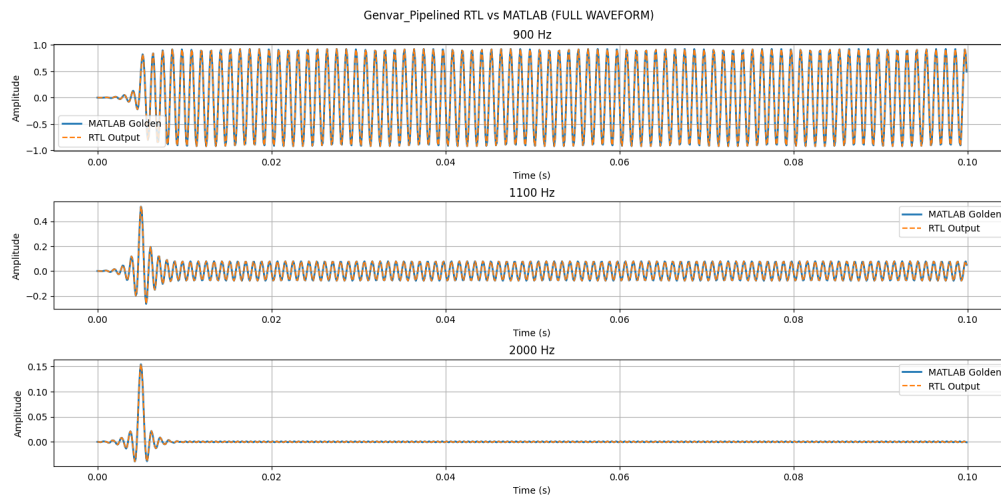


Figure 2: Pipelined Symmetry-Based FIR using Adder Tree

6.3 Optimised Symmetric FIR

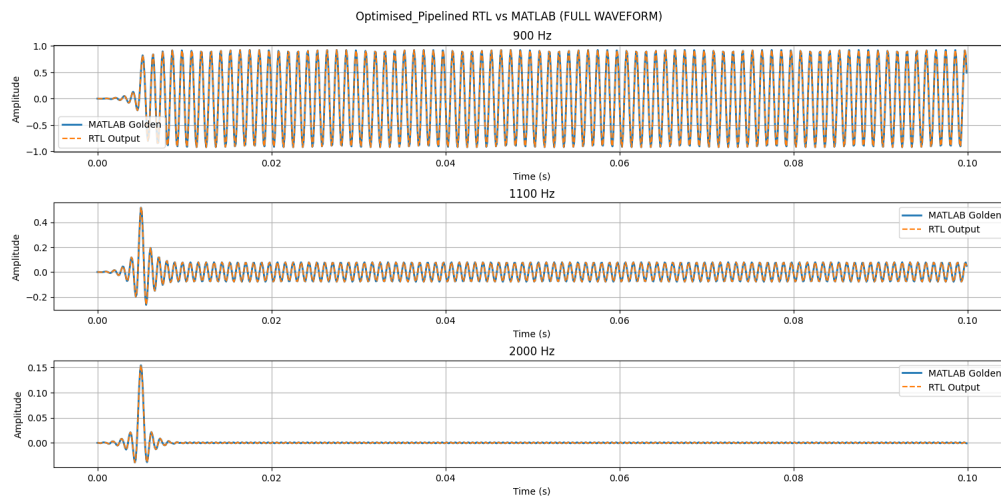


Figure 3: Optimised FIR Architecture with Sequential Accumulation

7 Simulation and Verification

7.1 Setup

- Simulation performed using ModelSim
- Inputs generated as multi-tone signals
- Outputs stored in fixed-point format

7.2 Verification Approach

- MATLAB used as reference model
- Outputs compared after latency alignment
- Waveforms validated for correctness

8 Resource Utilisation Analysis

Synthesis was performed using Intel Quartus Prime Lite.

8.1 Observation

- Pipelining significantly improves performance
- Area increases slightly due to registers
- Trade-off between latency and speed

9 Results and Observations

9.1 Quantitative Comparison

Table 1: Resource Utilisation and Performance Comparison

Metric	Direct (Pipelined)	Genvar (Pipelined)	Optimised (Pipelined)	Direct	Genvar	Optimised
ALMs	1869	1611	1611	1769	1003	1327
Registers	5774	4007	4007	1741	1741	1741
DSP Blocks	51	26	26	100	26	50
Fmax 1 (MHz)	257.14	190.29	226.6	55.83	60.89	58.17
Fmax 2 (MHz)	254.78	192.01	232.02	54.09	58.12	54.96

The optimised pipelined design achieves a balanced trade-off between area and performance, while the symmetry-based design significantly reduces DSP usage without major performance degradation.

10 Conclusion

A 101-tap FIR low-pass filter was successfully implemented using multiple architectures and pipeline strategies.

-
- Fixed-point design is essential for hardware efficiency
 - Symmetry reduces computational complexity
 - Pipelining improves speed at the cost of latency